

1		Lv-3 Dragoon benchmark (KBSDL + IBSDL)	Unnamed: 1	Unnamed: 2
2	10		NaN	NaN
3	11	Sub-algorithm	mean time (ms)	stdev (ms)
4	12	AttKGen (KBS.KGen)	0.163121	0.015596
5	13	BlGen (KBS.BlGen)	0.010555	0.000413
6	14	BlPubKey (KBS.BlPubKey, pre-sampled bk)	0.562786	0.051079
7	15	BlSign (KBS.BlSign, precomputed bk,bpk)	0.176333	0.019885
8	16	Attest (BlGen+BlPubKey+BlSign, Fig.3)	0.712488	0.054265
9	17	AttVf (KBS.Vf)	0.71796	0.07231
10	18	VerKGen (IBS.Setup)	0.161126	0.017343
11	19	ProxKGen (IBS.Extract)	0.169166	0.019318
12	20	IBS.Sign (Fig.5, rk+message only)	0.334219	0.039875
13	21	ProxSign (AttVf + IBS.Sign)	1.037927	0.067041
14	22	VerSign (AttVf + Extract + IBS.Sign)	1.213079	0.093376
15	23	IBS.Vf (Fig.5, mpk+id+message only)	1.243106	0.128391
16	24	FinVf (AttVf + IBS.Vf)	1.921076	0.085893
17	25	End-to-end (AttKGen+VerKGen+Attest+VerSign+FinVf)	4.1875	0.171949

为了让你在论文写作或做 PPT 汇报时能清晰地把握理论流程和实验数据对应起来，我将以论文中的 **Lv-3 Dragoon** 流程为主线，一步步拆解，并直接映射到你 Excel 表格（以 P-256 为例）中测量的对应字段。

Lv-3 Dragoon 旨在实现：设备（Attester）匿名生成证明 → 验证者（Verifier）授权代理 → 代理（Proxy）跨平台代签 → 最终验证。

0.1 第一阶段：系统初始化与设备注册（部署期）

这一阶段发生在系统运行之前，主要涉及密钥的生成。

- 步骤 1：证明者生成密钥 (Attester Key Generation)
 - 理论过程：设备（如树莓派）生成自己的底层身份私钥 sk_a 和公钥 pk_a 。这是最原始的身份，绝对不能对外泄露。
 - 对应测评字段：AttKGen (KBS.KGen)
 - P-256 耗时：0.163 ms
- 步骤 2：验证者建立系统主密钥 (Verifier Setup)
 - 理论过程：验证中心生成全局主私钥 msk （即 sk_v ）和主公钥 mpk （即 pk_v ）。
 - 对应测评字段：VerKGen (IBS.Setup)
 - P-256 耗时：0.161 ms

0.2 第二阶段：设备匿名证明生成（高频运行期，最关键的隐私部分）

当设备需要证明自己环境安全时，它不能用 pk_a 签名，否则会被追踪。

- 步骤 3：生成盲化因子 (Blinding Key Generation)
 - 理论过程：设备内部生成一个随机数 bk （盲化因子 β ）。
 - 对应测评字段：BlGen (KBS.BlGen)
 - P-256 耗时：0.010 ms（因为纯生成随机数，不涉及点乘，所以极快）
- 步骤 4：计算盲化公钥 (Blinded Public Key)

- **理论过程**：设备利用 bk ，把自己的原始公钥“乔装打扮”成一个新的盲化公钥 $p\hat{k}_a$ 。外界无法把 $p\hat{k}_a$ 关联回 pk_a 。
- **对应测评字段**：BlPubKey (KBS.BlPubKey, pre-sampled bk)
- **P-256 耗时**：0.562 ms
- **步骤 5：使用盲化密钥签名 (Blinded Signature)**
 - **理论过程**：设备用私钥 sk_a 配合盲化因子 bk ，对 TEE 报告 m 进行签名，生成证据 σ_a 。
 - **对应测评字段**：BlSign (KBS.BlSign, precomputed bk,bpk)
 - **P-256 耗时**：0.176 ms
- **💡 综合阶段二：完整证据生成**
 - 以上三个步骤（步3+4+5）合在一起，就是设备端暴露给外部的 **Attest** 接口，它输出了匿名的证据元组 $(p\hat{k}_a, \sigma_a)$ 。
 - **对应测评字段**：Attest (BlGen+BlPubKey+BlSign, Fig.3)
 - **P-256 耗时总计**：0.712 ms

0.3 第三阶段：代理授权与跨平台代签（代理期）

这一阶段引入了代理服务器（RA Service），用于解决异构 TEE 验证的兼容性。

- **步骤 6：代理权限派发 (Proxy Key Extraction)**
 - **理论过程**：验证者收到设备的匿名证据后，把盲化公钥 $p\hat{k}_a$ 直接当作一个“身份 ID”，通过 IBS 机制为代理服务器派生出一个专用的代理私钥 rk 。这保证了代理只能处理这一个特定的匿名设备，不能越权。
 - **对应测评字段**：ProxKGen (IBS.Extract)
 - **P-256 耗时**：0.169 ms
- **步骤 7：验证证据与代签 (Proxy Signing)**
 - **理论过程**：代理服务器拿到 rk 后，首先必须验证设备发来的证据是否合法（调用 **AttVf**）。确认合法后，代理利用 rk 为该证据打上最终的跨平台烙印，生成最终签名 σ_{fin} 。
 - **对应测评字段**：
 - 仅仅检查设备证据是否合法的耗时：AttVf (KBS.Vf) (0.717 ms)
 - 仅仅代理签名的耗时：IBS.Sign (Fig.5, rk+message only) (0.334 ms)
 - 代理端完整操作耗时：ProxSign (AttVf + IBS.Sign) (1.037 ms)

0.4 第四阶段：最终验证 (Final Verification)

最终的业务系统（或依赖方）需要验证整个链路是否可信。

- **步骤 8：联合验证**
 - **理论过程**：验证方拿到最终签名 σ_{fin} 、设备的盲化公钥 $p\hat{k}_a$ 和验证者的主公钥 pk_v 。它需要同时确认两件事：1. 底层设备的签名是有效的；2. 代理（代表验证者）的代签也是有效的。
 - **对应测评字段**：
 - 仅验证代理签名是否合法的耗时：IBS.Vf (Fig.5, mpk+id+message only) (1.243 ms)
 - 整体最终验证耗时：FinVf (AttVf + IBS.Vf) (1.921 ms)

0.5 总结：性能数据背后的故事

通过将理论流程映射到你的数据，可以得出一个非常漂亮的结论：

即使在拥有严密隐私保护（KBS）和跨平台代理委托（IBS）的 Lv-3 完整形态下，包含全部声明周期的**端到端开销（End-to-end）**在 P-256 下也仅仅需要 4.18 ms。对于大规模异构物联网设备的高频证明场景，这种微秒级到个位数毫秒级的开销是非常卓越的。